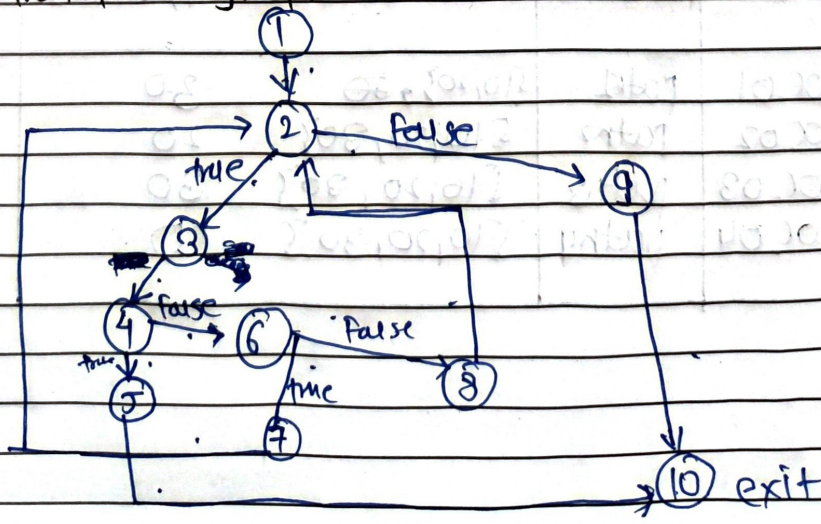
* program:- Binary Search.

```
int binarySearch(int arr[], int n, int target)
{
int low=0, high=n-1;                    // N1
while (low<=high)                       // N2
{                                       // N3
  int mid = low + (high-low)/2;         // N4
  if (arr[mid] == target)               // N5
      return mid;
  if (arr[mid] < target)                // N6
      low=mid+1;                        // N7
  else
      high = mid-1;                     // N8
}
return -1;                              // N9
}                                       // N10
```

① Control flow graph.



② Cyclomatic complexity

$= E - N + 2P$
$= 12 - 10 + 2 * 1$
$= 4$

Result: the complexity is 4 meaning we need four independent paths to achieve full branch coverage.

③ path exercised [control flow independent path]:

path1 : 1 → 2 → 9 → 10
path2 : 1 → 2 → 3 → 4 → 5 → 10
path3 : 1 → 2 → 3 → 4 → 6 → 7 → 2 ...
path4 : 1 → 2 → 9 → 4 → 6 → 8 → 2 ...

test case table :

ID	path	input data (array, target)		exp op	Act op
tc_01	path1	{10,20}, 30	30	-1	-1
tc_02	path2	{10,20,30}	20	1	1
tc_03	path3	{10,20,30}	30	2	2
tc_04	path4	{10,20,30}	10	0	0

* Data Flow Technique.

(i) Variable use

low, high, mid, target

(ii) table of Def-use (DU) pairs.

Variable	defined at node	used at node	type of use
low	1	2	predicted use
low	7	2,3	computation use
high	1	2	predicte use
mid	3	4,5,6	computation / predicted use

(iii) test case table (Data Flow)

ID	Variable packed	ip	DU path	Reason
WDO1	low	low=0	1→2	boundary checked.
WDO2	mid	target=20	3→4	mattered (correct)
WDO3	high	high=n-1	1→2	loop condition checked
WDO4	low	low=mid+1	8→2	next iteration, right subarray.
WDO5	high	high=mid-1	8→2	next iteration left subarray.

faq

(1) →

blackbox testing	white box testing
- Focus on functional requirements and specifications.	- Focus on logic, code structure and flow.
- internal code is not visible to tester	- internal code is fully visible.
- Eg. ECP, BVA, cause effect graphing	- control flow, data flow, loop testing.

(2) →

cyclomatic complexity

V(G) = P+1
= 3 decision nodes + 1 = 4

test cases :

ID	input	path	output
TC01	n=2	1→2(F)→6(T)→7→8	Number is prime
TC02	n=4	1→2(T)→3(T)→4→2(F)→8	Number is not prime
TC03	n=7	1→2(T)→3(F)→5→2(F)→ 6(T)→7→8	Number is prime

(3)

Variable	defined at	used at
n	scanf	temp = n
n	temp = n	while (n>0)
n	while loop	r = n%10
n	n = n/10	while (n>0)

test cases:-

ID	input	DU path	OP
TC01	n = 153	1 → loop → exit	armstrong number
TC02	n = 10	1 → loop → exit	not armstrong number

(4) → test cases are developed by testing the loop at its boundaries:-

i) skip loop : zero iteration
ii) Single pass : exactly one iteration
iii) avg pass : m iteration (where m < max)
iv) boundry pass : n-1, n, n+1 iteration (near maximum).

(5) → compound predicate testing

IF(age<65 and married == true)

① simple decision coverage : test cases to make

the whole IF result in true and false.

② conditional coverage:
test cases written each individual condition (operands and nonmicd) evaluated true & false.

③ decision-condition coverage:
combines both; ensure every condition and the final decision are tested for all possible outcomes.

⑥ →

Static testing	Structural testing
i] Done without executing code	i] done by executing code.
ii] Reviews, walkthrough, inspection	ii] executing paths, branches, and loops.
iii] Find errors in documentation/ logic	iii] Verifying internal logic and flow.

⑦ → test coverage criteria:

i) Statement coverage: every node in the graph is executed at least once.
ii) branch/decision coverage: every edge (true/false) is executed at least once.
iii) path coverage: every independent path from start to end is executed.